Se pide desarrollar una aplicación Swing para gestionar los **pedidos de una tienda**.

Modelo (3)

La aplicación va a trabajar con las siguientes clases:

- **Cliente:** id, nombre, email, teléfono, lista de pedidos
- **Pedido:** id, fecha, cliente, descripción, cantidad, precio unitario, pagado

Como se puede ver hay una relación **1 a N** entre clientes y pedidos.

En la clase **Pedido** se pide añadir el siguiente método:

- **double getImporte()** : El resultado de multiplicar la cantidad por el precio unitario

En la clase **Cliente** se pide añadir el siguiente método:

- **double getImporteTotal()** : El resultado de **sumar** los **importes de todos los pedidos** del cliente

ClienteDAO / ClienteImpl (5)

Se definirá un DAO con los siguientes métodos y en la implementación se almacenarán los **clientes** en un **ArrayList** estático llamado **clientes**. Métodos:

- **Cliente nuevoCliente(Cliente nuevo)** : En la implementación, este método
  - Se asignará al cliente el **siguiente ID disponible**
  - Se añadirá el cliente a la lista de clientes
  - Se retornará el cliente
- **Cliente getCliente(Integer id)** : Este método buscará el cliente con el id indicado en la lista de clientes. Si **existe** se **retornará**, si **no existe** se retornará **null**
- **List<Cliente>getClientes()** : Retornará la lista de clientes
- **void modificarCliente(Cliente modificar)** : Este método **sustituirá** el **cliente con el id** de **modificar** con el cliente **modificar**
- **void eliminarCliente(Integer id)** : Este método **eliminará** el cliente con el **id** indicado de la lista

PedidoDAO / PedidoImpl (2)

Similar al anterior (los pedidos se almacenarán en una lista llamada **pedidos**). Dispondrá de los siguientes métodos:

- **Pedido nuevoPedido(Pedido nuevo)** : Este método
    - Asignará a **nuevo** el **siguiente id** disponible
    - Añadirá el pedido al final de la lista
    - **Añadirá** el pedido a la **lista de pedidos del cliente**
    - **Retornará** el pedido
- **void pagarPedido(Integer id)** : Buscará el pedido con el **id** indicado y, **si existe**, pondrá su campo **pagado** a **true**

PedidoServicio (3)

La clase **deberá implementar el patrón Singleton**, definir los **dos daos anteriores** como **propiedades** de la clase e **implementar todos los métodos de ambos daos** (con los **mismos nombres**):

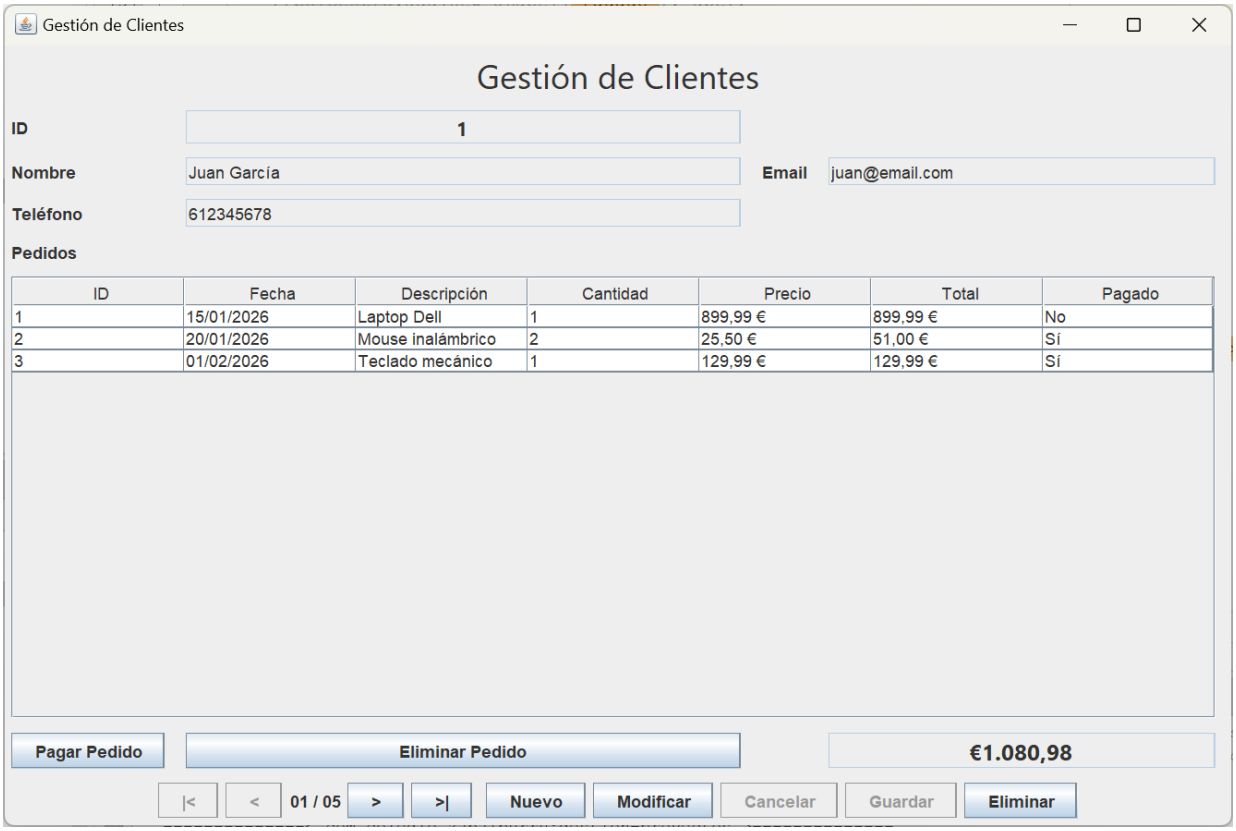
Adicionalmente, deberá definir el siguiente método:

- **List<Pedido> getPendientes()** : Retornará una lista con **todos los pedidos pendientes** de pago (campo **pagado** es **false**) ordenados en **descendente** por fecha
- **List<Cliente> getClientesSinPedidos()** : Retornará una lista con todos los clientes que **no han hecho ningún pedido** ordenados **alfabéticamente** por su nombre

Podéis emplear el **código al final del enunciado** para añadir algunos clientes y pedidos. **OJO!!**: Para que funcione, los métodos del dao y del servicio se deben llamar **exactamente** como aparece en el enunciado.

Vista (15)

Se pide diseñar un formulario con un aspecto similar al siguiente (en modo CONSULTA):



El formulario tendrá tres estados **ALTA, MODIFICACION** y **CONSULTA** e inicialmente estará en modo consulta.

- El formulario mostrará la información **en función del estado y de la información del cliente actual**. **NOTA**: Si **no hay clientes** nos pondremos en modo **ALTA**
- La tabla, en modo **CONSULTA** y **MODIFICACION** deberá mostrar la información de los **pedidos del cliente activo**. En modo **ALTA** deberá aparecer **vacía**.
- Los datos de la tabla aparecerán **correctamente formateados**
- Debajo de la tabla y en un **JFormattedTextField no editable** se mostrará el **importe total de los pedidos del cliente**
- Los botones de **desplazamiento** sólo estarán operativos en modo consulta y del cliente actual y actualizarán la posición actual en función de la posición del cliente actual
- Se mostrará en una pequeña etiqueta la **posición del cliente actual** (empezando a contar **desde 1**) **/** el **total de clientes**
- Los botones **Nuevo, Modificar** y **Cancelar** activarán el estado adecuado y actualizarán el formulario
- El botón **Guardar**:
    - Si es un **ALTA** creará un **nuevo Cliente**
    - Si es una **MODIFICACION** recuperará el cliente actual
    - En ambos casos, actualizará la información del cliente con los datos que se muestran en el formulario
    - En función del estado, llamará al método apropiado del servicio
    - Se pondrá en modo **CONSULTA** y actualizará la información del formulario
- El botón **Eliminar**:
    - Si el cliente **tiene pedidos** dará un **mensaje de error** indicando que no se puede eliminar clientes con pedidos
    - Si **no los tiene**, se mostrará un diálogo de **confirmación** para asegurarnos de que el usuario quiere eliminar el cliente actual
        - Si confirma la eliminación, se eliminará el cliente seleccionado empleando el método apropiado del servicio
        - Se actualizará el formulario **asegurándose de que la información se muestra correctamente** (mostrando el siguiente cliente al eliminado si lo hay y, si era el último, mostrando el último cliente)
- El botón **Eliminar Pedido**:
    - Si **no hay ninguna fila seleccionada en la tabla** le pedirá al usuario que seleccione el pedido que quiere eliminar
    - Si la **hay**
        - recuperará el **pedido que está en la misma posición que la fila seleccionada**
        - pedirá **confirmación** y, si se da:
            - **eliminará** el pedido con el método adecuado del servicio
            - **actualizará** la información del formulario
- El botón **Pagar Pedido**:
    - Si **no hay ninguna fila seleccionada en la tabla** le pedirá al usuario que seleccione el pedido que quiere eliminar
    - Si la **hay**
        - recuperará el **pedido que está en la misma posición que la fila seleccionada**

- si el **pedido ya está pagado** dará un mensaje al usuario indicando esta circunstancia
- si **no lo está**, se pagará el pedido con el método correspondiente del servicio y se actualizará el formulario

Inicialización de datos (sólo para el primer cliente)

Método para llamar **desde el constructor** de la clase de **Servicio**:

```
public void crearDatos()

    nuevoCliente(new Cliente(1,"Juan García",juan@email.com,"612345678"));

    nuevoCliente(new Cliente(2,"María López",maria@email.com,"623456789"));

    nuevoCliente(new Cliente(3,"Carlos Rodríguez",carlos@email.com,"634567890"));

    nuevoCliente(new Cliente(4,"Ana Martínez",ana@email.com,"645678901"));

    nuevoCliente(new Cliente(5,"Pedro Sánchez",pedro@email.com,"656789012"));


    Pedido pedido1 = new Pedido(null, LocalDate.of(2026, 1, 15), getCliente(1),
"Laptop Dell",1, 899.99);

    nuevoPedido(pedido1);

    Pedido pedido2 = new Pedido(null, LocalDate.of(2026, 1, 20), getCliente(1),"Mouse
inalámbrico", 2, 25.50);

    nuevoPedido(pedido2);

    Pedido pedido3 = new Pedido(null, LocalDate.of(2026, 2, 1), getCliente(1),
"Teclado mecánico", 1, 129.99);

    nuevoPedido(pedido3);

}
```